# 面向 Cyclone IV 的 Yosys 逻辑综合面积优化与等价验证


## 摘要

针对 FPGA 综合中译码选择、常量查表与寄存器控制逻辑在 Cyclone IV 上映射后仍存在冗余的问题，本作品在 Yosys 中实现了一组结构受限、收益可结算的改写规则，并把它们接入 synth_intel 的既有流程：在 RTLIL 层识别分支条件常量、完整或显式部分的等值译码与常量 ROM；对优先选择链一类输入，由统一映射流程处理；在映射阶段把细粒度寄存器组合到器件专用同步控制，并把只驱动一个寄存器数据端的 LUT 吸收进同步装载；在 ABC9 映射后对 LUT4 做有界局部重写与输入排列规范化，折叠被使能逻辑占用的 LUT，并统一使用同一套 ABC9 面积脚本。九个赛方用例（4 个基础用例、5 个压力用例）按本文口径统计的逻辑单元（cycloneiv_lcell_comb 与残留 $not 之和）由 2399 降至 1741，逐例降幅平均 26.55%，单例最高 36.11%、最低 20.00%，DFF 均未增加；78 个设计的两个检查点共 156 项独立证明、28 个新增设计的 RTL 归纳证明与 48 种启动配置全部通过。正式效率批次的经过时间开销为 −10.70% 至 +3.71%，主进程峰值工作集变化为 −6.28% 至 +2.13%，增幅均不超过 5%，属本次正式批次的观测值；内部 3% 余量目标未全部达到，短用例的时间波动仍然存在。另以一个自建的千单元级混合状态电路做压力验证，逻辑单元由 1016 降至 754（−25.79%；另一参考构建为 999 降至 754，−24.52%），DFF 803→800。


关键词：FPGA；Yosys；逻辑综合；多路选择器；等价验证

## 第一部分 作品概述

### 1.1 功能与特性

本作品是一组面向 Cyclone IV 的开源综合优化能力，以独立 pass 的形式编入 synth_intel，使用者执行原有综合命令即可获得，无需改动设计源码或外部脚本。能力分为三层：RTLIL 结构层改写选择与译码网络，包括分支常量传播、译码表重构与共享、常量 ROM 共享以及受控的公共算子外提；触发器层把共享时钟、复位与使能的细粒度寄存器组合映射到器件专用同步控制，并把只驱

动一个寄存器数据端的 LUT 吸收进同步装载；LUT 层在 ABC9 映射后对有界窗口做精确重写、对真值表做输入排列规范化，并折叠被使能逻辑占用的 LUT。三层规则采用相近的保守原则，但各自的适用条件与搜索限制独立设置：有的限制累计工作量，有的限制候选数与信号规模，排列规范化则限制缓存种类与访问次数。任何一条规则在前置条件不成立时都保留原电路，不把“未识别到风险”当作“证明可改写”。

### 1.2 应用领域

本作品适用于以 Intel（Altera）Cyclone IV 为目标、使用 Yosys 完成综合的 FPGA 设计，尤其是控制通路中大量使用 case/if-else 译码、优先 MUX 链、常量 ROM 与共享使能寄存器的场合。规则中与器件相关的部分（上电初态策略、同步控制映射、原语映射）依赖 Cyclone IV 的原语语义，迁移到其他器件族时必须重新核查这些语义，不能直接沿用。

### 1.3 主要技术特点

（1）按结构识别，不按名字匹配。规则只检查单元类型、位宽、驱动关系与真值表，同一版本对所有输入使用同一套规则、同一套映射脚本，不依据文件名、模块名或资源计数指纹分流。

（2）收益在最终映射上结算。局部代价与结构代价用于候选筛选和规则内的提交判断；整版候选是否最终采用，由综合之外的验收环节按最终 Cyclone IV 网表的逻辑单元与 DFF 计数判定，避免出现“局部变小、整体变大”的情况。

（3）保护边界显式。keep 与 init 属性、未知单元、黑盒、多驱动、部分选择、异步装载与复合置位清零都在规则入口处被识别为边界并跳过；预算耗尽时保留已提交的改写，剩余对象保持原样。

（4）改写与证明分离。候选网表必须通过独立工具的等价证明才被认定为可用；证明不通过时拒绝该映射结果，而不是修改证明配置、追加复位条件或更换输入。

（5）效率与正确性分别计量。时间与内存按同一批次、同一预热与复核规则采集，正式批次与稳定性观察分别报告，不用较好的一次覆盖较差的一次。

## 1.4 主要性能指标

表 1 九个赛方用例的主要指标

| 指标 | 九个赛方用例的结果 |
| --- | --- |
| 逻辑单元合计 | 2399 → 1741 |
| 逐例降幅 | 平均 26.55%，最低 20.00%，最高 36.11% |
| DFF 数量 | 九例均未增加（test3 由 17 降至 14） |
| 两个检查点等价 | 9 例 × 2 检查点，全部通过 |
| 扩展回归 | 78 个设计 / 156 个检查点全部通过 |
| 正式批次经过时间开销 | −10.70% 至 +3.71% |
| 正式批次主进程峰值工作集开销 | −6.28% 至 +2.13% |

## 1.5 主要创新点

一是把“结构识别—候选筛选—最终映射结算—独立证明”串成一条可复现的链路，使完整优化结果在最终映射上结算，并能被独立证明核查；二是把目标器件的专用资源（同步清零与装载控制、LUT4）作为改写目标而不是事后补救，使规则与目标器件的能力对齐；三是在同一套流程内同时约束面积、等价性与运行开销，任一维度失败都能把范围缩小到具体设计或检查点，为进一步定位规则提供依据，而不是只能笼统地怀疑整个流程。

## 1.6 设计流程

输入 Verilog 经前端解析为 RTLIL，先执行结构改写，再进入目标器件映射，最后做映射后的局部优化；原版与优化版共用同一份源码基线、同一套编译工具链、同一个 ABC 程序与同一套统计口径。等价验证、资源导出与效率计量彼此独立运行，互不干扰。系统组成与数据流见图 1。

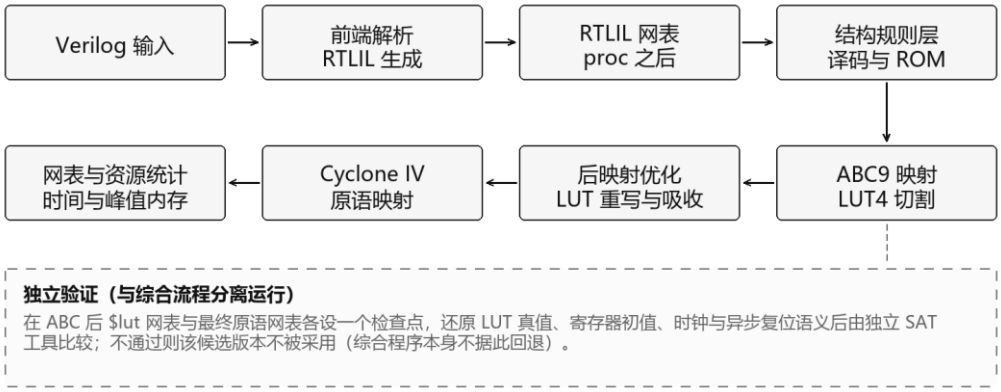



图 1 综合工具的系统组成与数据流

# 第二部分 系统组成及功能说明

## 2.1 整体介绍

系统由前端解析、结构规则层、ABC9 映射层、后映射优化层、Cyclone IV 原语映射与独立验证器组成。前五层都在 Yosys 进程内完成，验证器是独立程序，不参与综合，也不依赖综合程序的内部运行状态，只读取导出的网表。各规则在 synth_intel 脚本中的插入位置见图 2：结构规则在 proc 之后、flatten 之前；上电初态策略只在结构改写确实命中时插入；触发器原语的映射在 ABC9 之前的 map_ffs 阶段完成，图中为简洁起见按主数据流合并绘制；LUT 层规则位于 map_luts 的 ABC9 之后；原语映射位于 map_cells。

## 2.2 硬件系统介绍

### 2.2.1 硬件整体介绍

实现载体是综合软件与规则源码，目标器件为 Cyclone IV。评价对象是综合后的映射网表逻辑单元与 DFF 计数，以及同一次运行的完整经过时间与主进程峰值工作集；评价不包含开发板实测，也不包含布局布线后的时序与功耗。

### 2.2.2 机械设计介绍

本项目为综合软件与规则实现，不涉及机械结构、外壳、安装机构或机械加工设计。

### 2.2.3 电路各模块介绍

test1 是状态机控制的多通道数据选择电路，由状态寄存器与组合译码选择构成，主要检验译码表重构。test2 是四路 5 位优先选择电路，主体是纯组合 MUX 链，主要检验统一映射流程对优先选择链这类纯组合选择网络的化简。test3 是串行移位与命令控制电路，含寄存器、复位与命令译码，主要检验上电初态与同步控制映射。test4 是 5 位输入到 6 位常量编码与标志生成电路，主要检验映射后的 LUT 重写。pmux_case01 至 pmux_case05 是压力用例，分别覆盖均衡译码、流水化选择、交叉开关、反馈结构与嵌套选择，用于检验规则在更宽、更深、含反馈的选择网络上是否仍然稳定。

### 2.2.4 自建压力电路

官方用例的规模都在数百单元以内，结构也相对单一，难以检验规则在混合结构、深状态与反馈回路上是否仍然成立。为此我们另写了一个约千单元规模的压力电路用于自检。它不属于评分用例，只作为补充证据。

该电路把五个选择器用例与四个基础用例混排进同一个模块，并刻意加入三类脚手架：其一，16 个自启动的 32 位线性反馈移位寄存器（LFSR） 产生 512 位伪随机激励，大幅降低输入退化为常量、整块子电路被常量传播折叠的可能；其二，16 位周期计数器与 8 位相位计数器合成握手信号，为需要有效位、片选、使能与 ROM 地址的用例提供有意义的时序；其三，64 位多输入签名寄存器（MISR）吸收全部被测输出，减少未被观测逻辑被当作死逻辑删除的可能。此外还有一条由“三输入异或加触发器”构成的校准链，用于把规模精确调到目标值，链尾接到独立的探测输出，防止它被并入观测逻辑。

复位采用异步方式：Cyclone IV 的触发器原语本身带异步复位与使能端，控制逻辑可以被吸收进专用端口，但状态寄存器仍然占用 DFF 资源。同步复位并非必然多花逻辑单元——本作品的同步控制映射正是把这类控制折进 SCLR/SLOAD，因此这里只把异步复位当作该电路的构造选择，而不写成器件层面的普遍代价。电路输入只有时钟、复位与使能，输出为 64 位签名与 32 位探测，共 96 位公开输出；在当前参数下，两份原版参考分别给出 1016 与 999 个逻辑单元（均为 803 个 DFF），两个分母的处理见 3.3.5。

这样设计的用意是提高错误的可观测性：局部错误更容易在签名或探测输出上暴露，从而在自检中被注意到。需要说明的是，MISR 是压缩观测，存在不同错误相互抵消的可能，因此它只用于增加可观测性，不能代替等价证明；功能结论仍以完整状态关系证明为准。




图 2 各规则在 synth_intel 中的执行位置

## 2.3 软件系统介绍

### 2.3.1 软件整体介绍

规则按作用位置分为结构层、触发器层与 LUT 层三组，各自有独立的识别条件、保持关系与退出条件。表 2 汇总了各规则模块的识别条件与保留策略，详细条件以实现代码与帮助信息为准。

表 2 规则模块的识别条件与保持关系

| 规则（实现） | 识别条件 | 成功动作 | 保留或跳过 |
| --- | --- | --- | --- |
| 分支常量传播 pmux_branch_const | 等宽二值相等比较、唯一驱动、已知常量，作用于 $pmux 选择位 | 在该分支成立的前提下替换该分支数据中对应的位 | keep、未知值、歧义驱动跳过；预算耗尽时保留原电路 |
| 译码表重构 pmux_decode_tree | 1–5 位完整或显式部分的等值译码域 | 以共享子结构重建选择树，缺失地址沿用原默认输入 | 不完整且未开启部分域时保留；共享区域整体保留 |
| PMUX 公共算子外提 pmux_xor_factor | 分支互斥可证的 $pmux 区域，各分支数据由同类运算（异或或加减）驱动 | 把运算移到选择之后只做一次；异或要求共享操作数，加减则分别选择两侧 | 无法证明互斥、类型或位宽不一致、分支运算被多处使用时跳过 |
| 常量 ROM 共享 rom_share | 2–32 行、1–256 位、初始化完整、单次异步读且无写 | 重构为共享决策图 | 未知位、写端口、不支持的控制跳过 |
| 同步控制映射 map_sync_controls | 共享时钟、复位与使能的细粒度寄存器组 | 映射到 SCLR/SLOAD 专用控制 | keep、歧义 Q 驱动、未知控制跳过 |
| 寄存器装载吸收 lut_ff_load | LUT1–4 或标量 MUX 只驱动单个 dffeas 的数据端 | 用空闲 SLOAD/ASDATA 及可选 SCLR 吸收 | 必须净减少一个 LUT/NOT，初值与上电值保持 |
| 使能折叠 ff_enable_fold | 1–2 输入使能 LUT 仅被兼容的使能端使用 | 折入 ENA/SLOAD/数据端 | 要求 SCLR、SLOAD、ALOAD 空闲且 PRN 无效 |
| LUT 窗口重写 lut_window | 已知二值 LUT4，≤7 个切割叶、≤48 个内部 LUT、每根 ≤64 个 cut | 一到两个 LUT 的精确替换 | 严格减少 LUT、全图最大级数不增、图无环 |
| 输入排列规范化 lut_window 内 | 1–4 输入完全二值的 LUT | 同步置换输入与真值表，取数值最小的表 | 512 种缓存、16384 次访问上限 |

### 2.3.2 分支常量传播

多路选择器的每个选择位通常由一个比较器驱动，例如选择位由 X 等于 3 的相等比较产生。该分支被选中时 X 的取值已经确定，因此该分支数据中凡是引用 X 的位都可以直接替换为对应的常量位。规则逐个检查 $pmux 的选择位：若选择位由唯一驱动的等宽相等比较产生、比较对象是完全确定的二进制常量，就在“该分支成立”的前提下替换该分支数据中对应的位，默认输入、选择端与

其他分支保持不变。这一改写不引入任何分支互斥的新假设，只使用已有的选择语义，因此不会改变未被选中时的行为。

### 2.3.3 译码表重构与共享

译码表重构针对由一组互斥相等比较驱动的选择网络。当所有选择信号都来自同一控制向量的唯一译码、并且覆盖该向量的完整取值域时，整个选择网络等价于一张以控制向量为地址的表。规则以共享子结构重建这张表，把重复出现的比较与选择合并，减少为每个地址单独生成的译码逻辑。对不完整的译码域，本流程默认开启部分域支持（调用时带 -partial -vector -leaf 0 -rank 0），把缺失地址填成原默认输入，而不是填入任意无关值，这一点决定了改写后的表在未定义地址上仍与原件一致；若未开启该选项，不完整的译码域保留原电路。若某块区域内已有共享中间结果，则整块保留，避免按单表成本模型把跨表复用拆断。结构代价既参与候选排序，也参与是否提交的判断：只有新代价低于原代价，或在当前流程采用的部分域规范化配置下与原代价相等时才接受（该配置为 -partial 且 leaf=0，未启用 near 与 lut4）；候选是否最终保留，仍由验收环节按最终映射计数判定。

### 2.3.4 常量 ROM 共享

常量 ROM 在综合后表现为一个只读存储器与一次异步读。规则只接受行数为 2 至 32 的二次幂、位宽 1 至 256、初始化数据完整、无写端口、无复位与使能控制的 ROM，把它重构成共享决策图：地址位作为决策变量，数据位作为叶值，相同或相近的数据行共享同一条路径。搜索按叶宽与根到叶顺序进行，行访问次数、候选数与信号规模都有上限；超出上限的 ROM 保持原样，已生成的计划只有在完整搜索结束后才提交。

### 2.3.5 PMUX 公共算子外提

当一组互斥的 $pmux 分支对各自数据做同一种运算时，可以把运算移到选择之后只做一次：例如 MUX(X⊕A, X⊕B) 改写为 X⊕MUX(A,B)，选择控制端本身不变。对异或，要求所有分支共享同一个操作数；对无符号加减法，则分别选择两侧操作数，再执行一次同类型、同位宽的运算。规则只在分支互斥可证、运算类型与位宽一致、各分支运算被独占使用时才提交。该规则做的是结

构性合并，不在提交前运行完整映射，因此最终逻辑资源收益由综合后的验收确认；无法证明互斥、类型混杂或分支运算被多处使用时，保持原电路。

### 2.3.6 同步控制映射与寄存器装载吸收

Cyclone IV 的寄存器带有 SCLR、SLOAD、ASDATA 等专用控制端。若一组寄存器共享时钟、复位与使能，就可以把这类带同步复位与使能的触发器单元成组映射到专用控制端，数据输入可以不同：同步清零走 SCLR，同步置一走 SLOAD 并由 ASDATA 提供数据；时钟、复位、使能与装载的极性和优先级逐项保持。规则先按控制端分组，再核对整组的类型与控制端极性，只有整组都能映射时才提交，避免只改一半。

在此基础上，若某个 LUT 或标量 MUX 只驱动一个 dffeas 的数据端，且该寄存器还有空闲的 SLOAD/ASDATA（必要时加 SCLR），就把这一级逻辑吸收进同步装载，省掉一个物理 LUT 或反相器；吸收前要求使能与复位优先级不变、初值与上电值保持，并且每一步都必须净减少一个 LUT 或 NOT。使能折叠处理的是只被兼容使能端使用的 1 至 2 输入使能 LUT：把使能表达式折入 ENA、SLOAD 或数据端，要求 SCLR、SLOAD、ALOAD 空闲且 PRN 无效，并在全部数据与状态组合上逐项验证折叠前后一致。装载吸收在 ABC9 之前与之后各执行一次：前者处理原语映射后仍未被掩蔽的末端 LUT，后者处理 ABC9 新产生的同类结构。

### 2.3.7 映射后的 LUT4 有界重写

ABC9 映射之后网表由 LUT4 组成，局部仍可能存在可被更少 LUT 实现的冗余逻辑。窗口重写以每个已知二值 LUT 为根，向上展开不超过 7 个切割叶、不超过 48 个内部 LUT、每根最多 64 个 cut，在窗口内搜索用一到两个 LUT 在切割边界上重新实现根函数的方案。提交条件有三条：局部所有输入组合的真值完全一致；替换后 LUT 总数严格减少，既允许删除两个、增加一个，也允许删除三个、增加两个，净减少为正即可；全图最大 LUT 级数不增加，且改写后不出现环。状态单元、存储、黑盒、keep 与 init 信号都作为窗口边界，不参与重写。搜索有工作量上限，耗尽时保留已提交的改写，其余根保持原样。LUT 技术映射本身是 FPGA 综合的既有研究内容[4]，本作品的贡献在于把重写限制在有界窗口内并逐次结算收益。

### 2.3.8 输入排列规范化

同一逻辑函数在映射时可能因输入引脚顺序不同而被实现成不同模板，进而占用不同的 LUT 与内存。输入排列规范化对 1 至 4 输入、完全二值的 LUT 枚举输入位置排列，按同样的排列变换真值表，取数值最小的那张表作为规范形式；数值相同时保留原有排列，保证结果确定。变换只改输入端口顺序与真值表参数，不增删寄存器、不改输出连接、不插入反相器、不调整映射脚本。由于叶信号集合不变，组合依赖与逻辑深度也不变；正确性义务是所有二值输入赋值下输出相同，实现上对相同函数复用缓存，并对缓存种类数与访问次数设置上限。该规则是本版本新增的一项，目标不是继续降低面积，而是减少重复的映射模板与内存占用；在本版本的验收中，九例与压力电路的面积结果不因它而改变。

### 2.3.9 上电初态策略

在本文讨论的默认映射情形下，Cyclone IV 的触发器上电为低电平，而 RTL 中未指定初值的寄存器在语言层面是未知值，两者不能直接等同：把未指定初值一律补零会改变依赖初值的反馈电路行为，完全不处理又无法给出确定的映射语义。本作品只在前置探测接受分支常量或译码表改写路径时才执行统一的初值策略：对未指定初值的寄存器，把该状态位的逻辑初值设为它的异步复位值，没有异步复位的位设为 0；显式初值一律保留；异步装载、复合置位清零与未知单元不作推断，直接跳过。这里设置的是 RTLIL 状态位的逻辑初值；后续合法化可能改变极性，最终物理寄存器的上电值需结合映射解释。这样既保持了显式初值电路的行为，又避免对未复位启动的电路强加复位条件。触发器映射语义依赖目标原语[3]。

### 2.3.10 统一 ABC9 映射与平台适配

早期做法针对不同结构切换多套映射脚本，收益难以归因，也容易把映射器调参当成算法收益。本作品改为全流程单一脚本：所有输入、所有规则命中情况都使用同一套 ABC9 面积脚本，ABC 程序本身未修改，也没有用无关 pass 抵消新增开销。综合入口与脚本内容如下。

```
abc9 -lut 4 -W 300
script: +&sweep; &dc2; &dch -f -r -s; &ps; &if {W} {D} {R} -a -v
```

该脚本替换了上游 0.69 的内置默认脚本：上游默认串以 &scorr 开头、以 &mfs 结尾，&dch 只带 -f -r，且区分 default 与 area 两种；本文脚本去掉 &scorr 与 &mfs，给 &dch 增加 -s，并统一使用面积模式 -a。原版基线仍执行上游默认脚本，因此两臂的差异既包括规则，也包括这段脚本；这属于赛题答疑允许的“修改内置脚本”方式，不改变 ABC 程序，也不按用例切换。平台适配方面，ABC 的 -f 选项会用未加引号的路径拼出 source 命令，遇到含空格或中文的路径会失败。处理办法是把子进程的工作目录切到临时目录、只传 ASCII 文件名，并在所有脚本与临时文件路径上加引号；同时为进程启用 UTF-8 清单，临时目录改用原生接口删除，避免命令行删除在中文路径下的失败[7]。这些改动只影响路径传递与清理，不改变综合算法与结果。

### 2.3.11 验证接口

每个用例都在两个位置导出网表：ABC 之后、原语映射之前的 $lut 网表，以及最终的 Cyclone IV 原语网表。两侧使用同一套模型还原 LUT 真值、dffeas 的控制引脚与上电值，再由独立 SAT 工具逐检查点比较。含状态的设计比较全部寄存器初值、每个时钟与异步复位事件、下一状态以及公开输出；纯组合设计比较完整输入空间。证明不施加先复位条件，也不假定输入取值。需要说明证明对象：两个检查点比较的是映射网表本身，初值取自触发器的 power_up 参数，而不是 RTL 里声明的初值。对 RTL 中未指定初值的寄存器，本流程会按 2.3.9 的策略确定逻辑初值，因此这类设计的端到端语义对应关系不在双检查点证明范围内；它们只有在显式给出初值后才进入 RTL 归纳证明。

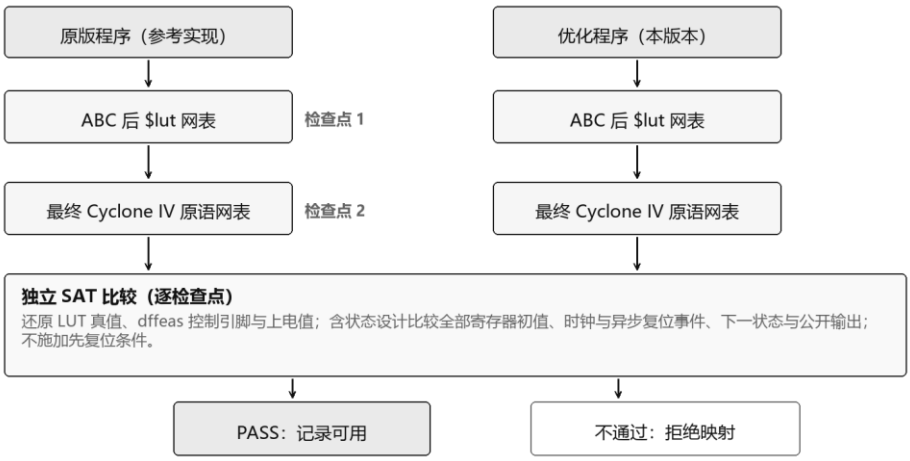



图 3 原版与优化程序的双检查点等价验证

这里有两层需要区分：综合过程中，单条规则只在结构前置条件成立、按规则自身的方式完成局部检查且未超出其搜索限制时才提交改写，条件不满足即保留原电路；整版候选是否可接受则由设计之外的验收环节判定，任一检查点不通过，该候选版本即不被采用，不会修改证明配置去迁就结果。

# 第三部分 完成情况及性能参数

## 3.1 整体介绍

原版与优化版使用同一份 Yosys 基线源码、同一套编译工具链与配置、同一个 ABC 程序；两臂的差异限于新增规则与该内置 ABC9 脚本。构建从新的空目录开始：配置 CMake、运行 Flex 与 Bison 生成 Verilog 解析器、由相应生成脚本产出模式匹配器与帮助数据，再以固定的优化选项编译全部 160 个对象并链接，不使用旧对象或旧生成物。候选程序哈希为 600cc3db…7c2f7b25，包内提供直接运行入口、网表严格复验入口与源码重建入口，运行依赖按相对路径解析。验证与效率测量在独立目录进行，不与构建并行。统计口径依据赛题指南与答疑[5-6]，测量方法沿用冻结的计时规则，未自行更改预热次数、运行顺序或复核条件。

## 3.2 工程成果

### 3.2.1 机械成果

本作品为综合软件与规则实现，不涉及机械实物，无机械加工成果。

### 3.2.2 电路成果

为九个用例分别保存了原版与优化版的 $lut 网表、最终原语网表与资源计数，可直接用于核查综合后的结构与面积变化，也可作为后续回归的对照基线。

### 3.2.3 软件成果

软件成果包括各规则源码、synth_intel 集成、可执行程序、构建记录、验证脚本与测量脚本。统一综合命令如下。

```
read_verilog TOP.v
synth_intel -family cycloneiv -top TOP
```

## 3.3 特性成果

### 3.3.1 实验配置与统计口径

逻辑单元记为 cycloneiv_lcell_comb 与残留 $not 之和，DFF 单独统计；逐例降幅为（原版数量 − 优化后数量）除以原版数量。面积、时间、内存三项分别评价，不用一项的收益抵消另一项的损失。

效率按冻结的计时规则测量：每例先对原版预热 10 次，再按原版、优化版的顺序各运行一次；若原版与优化版首轮耗时均不足 1 s 且任一指标超过 5%，则追加 5 组配对复核，判定值取“（优化版五次指标之和 ÷ 原版五次指标之和 − 1）×100%”，而不是五个百分比的算术平均；首轮数据同时保留、不删除，表中百分比按未舍入数据计算。经过时间用 Measure-Command 记录完整耗时（包含等待 ABC），内存按 1 ms 采样主进程峰值工作集。测量期间不运行构建与证明；系统后台与杀毒条件未修改，也未做独立认证。测试平台为单台 Windows 主机：Intel Core Ultra 7 255H（16 核）、31.4 GB 内存、Windows 11 家庭版（10.0.26300）。独立验证使用 OSS-CAD-Suite 自带的 Yosys（0.69+274，验证程序 SHA-256 为 bafe6f5c…）：映射网表由验证脚本还原成显式布尔方程与状态转移关系，RTL 归纳检查使用随实验保存的受限原语模型（primitive_decode.v），其支持范围由验证脚本检查；证明入口为包内 verify_functional.py。

### 3.3.2 逻辑资源与功能正确性

九个用例的最终映射资源与等价结果见表 3，降幅对比见图 4。逻辑单元合计由 2399 降至 1741，逐例降幅平均 26.55%，最低 20.00%（test2）、最高 36.11%（test1），九例均不低于 20%。DFF 方面，test1、test2、test4 与五个压力用例保持不变，test3 由 17 降至 14；该例以完整状态转移关系证明确认原版与优化版在初值、时钟与异步复位事件、下一状态上保持同一关系，没有把“寄存器数量相同”当作等价条件。

表 3 最终映射网表的资源（原版→优化，逻辑单元按本文统计口径）与等价结果

| 用例 | 逻辑单元 | 降幅 | DFF | 等价 |
|---|---|---|---|---|
| test1 | 288 → 184 | 36.11% | 222 → 222 | 通过 |

| 用例 | 逻辑单元 | 降幅 | DFF | 等价 |
|---|---|---|---|---|
| test2 | 25 → 20 | 20.00% | 0 → 0 | 通过 |
| test3 | 13 → 10 | 23.08% | 17 → 14 | 通过 |
| test4 | 34 → 24 | 29.41% | 10 → 10 | 通过 |
| pmux_case01 | 406 → 312 | 23.15% | 12 → 12 | 通过 |
| pmux_case02 | 444 → 297 | 33.11% | 24 → 24 | 通过 |
| pmux_case03 | 409 → 301 | 26.41% | 10 → 10 | 通过 |
| pmux_case04 | 365 → 285 | 21.92% | 16 → 16 | 通过 |
| pmux_case05 | 415 → 308 | 25.78% | 12 → 12 | 通过 |

各例的主要收益来源并不相同：test1 主要来自译码表重构，test2 来自统一映射流程对纯组合选择网络的化简，test3 来自同步控制映射与上电策略，test4 来自映射后的 LUT 重写；压力用例则在更宽的选择网络与反馈结构上同时触发多条规则。上述归纳来自规则日志与逐例对照，尚未对每条规则的贡献做单独消融，因此只作为定性说明，不给出定量拆分。

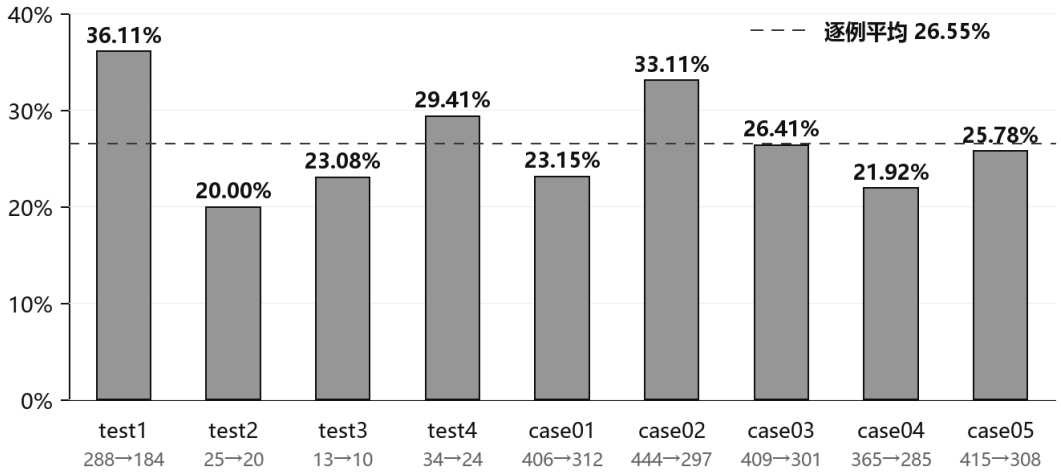



图 4 九个用例的逻辑单元降幅

### 3.3.3 时间与主进程峰值工作集

正式批次中，九例的经过时间变化为 −10.70% 至 +3.71%，主进程峰值工作集变化为 −6.28% 至 +2.13%，增幅均不超过 5%，逐例数值见表 4 与图 5。收益最明显的是 pmux_case02（−10.70%）与 pmux_case05（−7.68%），它们的共同点是改写后需要映射的选择网络显著变小；代价最大的是 test1（+3.71%），其

基线本身较短；该数值是完整流程的观测差异，包含运行波动，不宜直接视为规则自身的算法开销。

表 4 正式批次的效率开销（test1 取复核均值，其余取首轮值）

| 用例 | 原版/s | 优化后/s | 时间变化 | 主进程峰值工作集变化 |
|---|---|---|---|---|
| test1 | 0.2800 | 0.2904 | +3.71% | −6.28% |
| test2 | 0.1663 | 0.1622 | −2.44% | −0.54% |
| test3 | 0.1764 | 0.1719 | −2.54% | +1.02% |
| test4 | 0.1908 | 0.1841 | −3.55% | +2.13% |
| pmux_case01 | 0.3300 | 0.3175 | −3.80% | −0.30% |
| pmux_case02 | 0.3554 | 0.3174 | −10.70% | −4.42% |
| pmux_case03 | 0.3456 | 0.3314 | −4.10% | −0.48% |
| pmux_case04 | 0.3042 | 0.3054 | +0.40% | +0.67% |
| pmux_case05 | 0.3340 | 0.3083 | −7.68% | −5.79% |

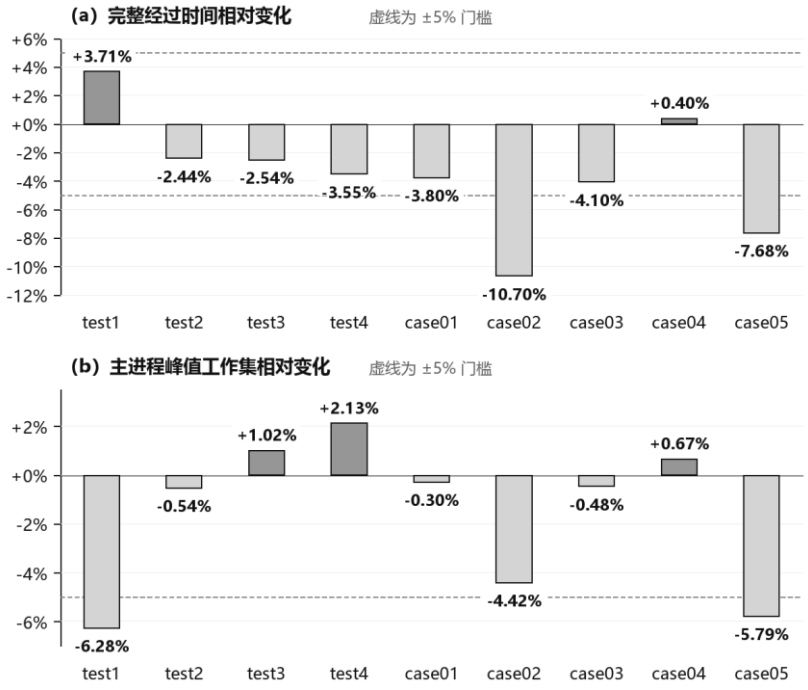



图 5 九个用例的时间与主进程峰值工作集开销（正式批次）

内部把“每例两项开销都不超过 3%”作为工程余量目标，本版本未全部达到：正式批次 test1 的时间为 +3.71%，第二组稳定性观察中 test3 的时间为 +4.26%。两组观察批次同时显示 pmux_case01 的峰值内存由早期贴近 5% 收敛到 −0.30%、−3.59%、−3.80%，说明该例的内存压力在当前实现下已留出余量；但跨批次差

异不能全部归因于代码，短用例的计时波动仍然存在，不能据此推断任意环境下都稳定满足 5%。

### 3.3.4 验证覆盖与适用范围

功能正确性由多层证据支撑，覆盖情况见表 5。78 个设计（含九个赛方用例）各设两个检查点，共 156 项独立证明全部通过，其中 36 个含状态设计做完整状态转移 SAT，42 个组合设计做完整输入空间 SAT；该批 156 项证明在候选与原版之间分段完成并串联，每段都比较完整的映射网表；新增的 24 个独立设计加 4 个重命名变体（共 28 个实例）完成从 RTL 到 ABC 后检查点的归纳证明，其中有状态实例均显式定义初值，全部没有逻辑资源反增；48 种启动配置覆盖有无初值、有无异步复位与两种时钟极性的组合；40 项局部 SAT 检查全部通过，另有两类故意注入的错误（初值错误、输出错误）被独立审计拒绝；输入排列规范化的缓存上限、访问上限与部分选择共 270912 项真值比较全部一致。自建压力电路与两份原版参考之间采用多对一状态关系：原版的每个寄存器都对应优化版的一个寄存器，允许多个原版寄存器对应同一个优化版寄存器，也允许极性取反；优化版的每个寄存器同样被覆盖。独立 SAT 在该关系的约束下检查两侧初值、时钟与异步复位事件、下一状态以及公开输出，因此覆盖的是全部满足该关系的状态，而不只是从某个初态可达的状态。其余含状态用例的双检查点关系由证明框架逐设计生成对应并独立核对，两侧寄存器数量可以不同，具体记录见证据目录。

表 5 验证覆盖与证据类型

| 证据类型 | 覆盖对象 | 结果 |
| --- | --- | --- |
| 双检查点等价证明 | 78 个设计，156 个检查点 | 全部通过 |
| RTL 归纳证明 | 24 个新增设计 + 4 个重命名变体 | 全部通过，无资源反增 |
| 启动配置 | 48 种（初值 × 复位 × 时钟极性） | 全部通过 |
| 局部 SAT 与错误注入 | 40 项局部检查、2 类故意错误 | 全部通过，错误被拒 |
| 真值表比较 | 缓存上限、访问上限、部分选择 | 270912 项一致 |

边界方面，证明只覆盖声明的离散二值模型：不覆盖任意 X/Z 事件调度、模拟上电过程与物理时序；全图最大 LUT 级数约束只是逻辑深度指标，不等于布局布线后的时序保证；LAB 打包、布线延迟与功耗未评价。原版默认 BRAM

流程在一个携带 INIT 参数的 M9K 用例上存在模板接口不匹配，该失败单独记录，双方改用 -nobram 的补充结果另行标注，不写作默认流程通过。适用范围与未覆盖项汇总于表 6。

表 6 适用范围与未覆盖项

| 项目 | 本版本的结论 |
| --- | --- |
| 面积 | 九例逐例不低于 20%，78 个已测设计无逻辑资源增加 |
| 功能 | 已测设计在两个检查点等价，含初值、时钟与异步复位事件 |
| 时间与内存 | 正式批次与两组观察均满足 5% 门槛；3% 内部目标未全部达到 |
| X/Z 与模拟语义 | 未覆盖 |
| 物理时序、功耗、布局 | 未评价 |
| 其他器件族 | 未验证，需重新核查器件相关规则 |

### 3.3.5 自建压力电路的验证结果

压力电路按与九例完全相同的流程与口径综合，结果见表 7。

表 7 自建压力电路的资源与等价结果

| 口径 | 逻辑单元 | 降幅 | DFF | 等价 |
| --- | --- | --- | --- | --- |
| 原版（同工具链重建） | 1016 → 754 | 25.79% | 803 → 800 | 通过 |
| 原版（另一构建） | 999 → 754 | 24.52% | 803 → 800 | 通过 |

同一份压力电路 RTL 在两份原版构建下分别得到不同的映射网表与逻辑单元计数（1016 与 999）。两者来自不同的原版构建，具体根因尚未定位，因此这里只按两个分母分别记录、不混用。优化版在两种口径下都是 754 个逻辑单元与 800 个 DFF：逻辑单元降幅分别为 25.79% 与 24.52%，DFF 减少 3 个，单次运行时间由 0.954 s 降到 0.926 s、峰值内存由 39.9 MiB 降到 38.0 MiB（该组效率数据对应同工具链重建参考）。两种口径下优化版的逻辑单元与 DFF 计数相同，并各自通过上述等价检查；本文没有保存逐文件网表哈希，因此不作“逐结构相同”的断言。降幅差异只来自分母，结论方向一致。

功能方面，两套参考各自在 ABC 后 $lut 网表与最终原语网表上做了完整状态关系证明，覆盖全部寄存器、实际初值、每个时钟与异步复位事件、下一状

态以及 96 位公开输出；证明未施加先复位条件。该电路只作为自检用例，不计入九例的平均降幅。

# 第四部分 总结

## 4.1 可扩展之处

后续工作集中在三处：一是把短用例的计时波动压到内部 3% 目标以内，并在独立环境复测，确认余量不是单机现象；二是把规则从逐条立项推进到组合验证，对多条规则同时命中的电路给出可分解的贡献度量；三是把验证入口前移到 RTL 端，明确未指定初值在 RTL 层的允许行为与参考器件实际行为之间的对应关系，使端到端证明有明确的语义基础。BRAM 厂商语义、双时钟结构与其他器件族的适配也需要单独立项。

## 4.2 心得体会

整个过程中最耗时的不是写出改写规则，而是确定“什么时候不该改”。早期为了追求面积，曾在结构条件不充分时也提出候选，结果在个别设计上出现回退，而且很难从最终网表反推原因。后来把每条规则的接受条件、保持关系与退出条件写成可审查的契约，并规定不满足前置条件就保留原电路，问题才变得可定位。

另一条经验是收益必须回到最终映射上结算。局部代价模型在多个用例上给出过与最终计数相反的结论，最终都以映射后的逻辑单元与 DFF 为准。此外，把证明与优化分开实现、分开运行，使失败范围能先收敛到某个设计或检查点，为进一步定位提供线索，而不是笼统地怀疑整个流程。

# 第五部分 参考文献

[1] YosysHQ. Yosys 0.69 documentation: Intel (MAX10, Cyclone IV)[EB/OL]. https://yosyshq.readthedocs.io/projects/yosys/en/v0.69/cmd/index_techlibs_intel.html.
[2] Berkeley Logic Synthesis and Verification Group. ABC: A System for Sequential Synthesis and Verification[EB/OL]. https://people.eecs.berkeley.edu/~alanmi/abc/abc.htm.
[3] YosysHQ. Yosys source code: techlibs/intel/common/ff_map.v[CP/OL]. commit 9f75ca1f9834a39a863915b5dae0c7b1e33533bc. https://github.com/YosysHQ/yosys/blob/9f75ca1f9834/techlibs/intel/common/ff_map.v.
[4] Cong J, Ding Y. FlowMap: An Optimal Technology Mapping Algorithm for Delay Optimization in Lookup-Table Based FPGA Designs[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 1994, 13(1): 1-12. DOI: 10.1109/43.273754.
[5] 中科亿海微电子科技（苏州）有限公司. 全国大学生嵌入式芯片与系统设计竞赛 2026 FPGA 创新设计赛道选题指南[Z]. 2026. 随赛题下发。
[6] 中科亿海微电子科技（苏州）有限公司. 全国大学生嵌入式芯片与系统设计竞赛 2026 FPGA 赛道选题二问题答疑汇总[Z]. 2026. 文件名：FPGA 创新大赛问题汇总 (1).xlsx，随赛题下发。

[7] Microsoft. Use UTF-8 code pages in Windows apps[EB/OL]. https://learn.microsoft.com/en-us/windows/apps/design/globalizing/use-utf8-code-page.

# 附录 A 提交包与复验说明

## A.1 包结构

表 A-1 提交包顶层条目

| 条目 | 内容 |
| --- | --- |
| yosys.exe | 综合程序，SHA-256 600cc3db…7c2f7b25 |
| yosys-abc.exe、share/ | 配套 ABC 与运行期数据、器件模型 |
| source/ | 完整优化源码与 abc9.cc 面积脚本 |
| verification/、evidence/ | 用例、期望网表、证明脚本与全部证据 |
| build-record/ | 构建命令、生成物与对象清单 |
| licenses/、README.md、manifest.json、SHA256.json、两个复验脚本 | 许可、说明、哈希清单与复验入口 |

## A.2 关键定位

ABC9 面积脚本作为内置字符串写在 source/passes/techmap/abc9.cc 的 constpad 中，全流程使用同一串，内容如下。

```
script: +&sweep; &dc2; &dch -f -r -s; &ps; &if {W} {D} {R} -a -v
```

功能复验入口为 python verify_functional.py --yosys <OSS-CAD-Suite/bin/yosys.exe> --out <新目录>，依赖合同固定的独立 Yosys 哈希；它重综合当前程序、重生成状态方程、对照两份原压力参考，并对 36 个显式初值的时序 RTL 或纯组合 RTL 用例进行等价证明，通过标志为 summary.json 的 status 等于 PASS_REEXPORTED_GRAPHS_FRESH_STATE_RELATIONS_BOTH_PRESSURE_REFERENCES_AND_36_RTL_CASES。映射网表由证明脚本还原成布尔方程与状态关系，RTL 归纳使用受限原语模型 primitive_decode.v，两条路径都只读导出网表。

整包校验执行 python verify_portable.py --check-only，按 SHA256.json 逐文件核对；建议解压到新目录复验。综合程序本身只读包内相对路径，不向工作目录写入产物。